



Text Representation & Feature Engineering



1. What is Text Representation?

Imagine you're talking to a computer 🖥️

You say:

"I love NLP"

But the computer says:

❌ "I don't understand words... give me numbers!"



So we convert text into numbers



In simple words:

Text Representation = **Converting words into numbers so machines can understand**



2. Why Do We Need This? (Problem → Solution)

● Problem:

Machines only understand:

- Numbers
- Vectors
- Mathematical patterns

But we give:

- Text
 - Sentences
 - Language
-

✅ **Solution:**

Convert text → numerical form

"I love NLP"

↓

[0.2, 0.8, 0.5]

Now machine can:

- Compare
 - Learn patterns
 - Make predictions
-

🧩 **3. Core Techniques (Your Topics)**

We'll go from **basic** → **advanced** 📌

◆ **1. Bag of Words (BoW)**

💡 **Idea:**

Count how many times each word appears

📦 **Example:**

"I love NLP"

Vocabulary:

[I, love, NLP]

Vector:

[I:1, love:1, NLP:1]

💡 **Analogy:**

Like counting items in a shopping list 🛒

⚠️ Limitation:

- Ignores word order
 - "I love NLP" = "NLP love I" ❌
-

◆ 2. TF-IDF (Smart Counting)

💡 Idea:

Not all words are equally important

📦 Example:

"The movie is good"

- "the", "is" → common → low importance
 - "movie", "good" → meaningful → high importance
-

💡 Formula Intuition:

- **TF (Term Frequency)** → how often word appears
 - **IDF (Inverse Document Frequency)** → how rare the word is
-

💡 Result:

Important words get **higher weight**

💡 Analogy:

Like highlighting important words in a book 📖

◆ 3. N-Grams

💡 Idea:

Capture word sequences (context)

📦 Example:

"I love NLP"

- Unigram → ["I", "love", "NLP"]
 - Bigram → ["I love", "love NLP"]
 - Trigram → ["I love NLP"]
-

💡 Why Important?

- Captures meaning:
 - "not good" ≠ "good"
-

💡 Analogy:

Understanding phrases instead of single words 🧩

◆ 4. Word Embeddings (Most Important)

💡 Idea:

Convert words into **dense vectors with meaning**

📦 Example:

king → [0.25, 0.78, 0.11]

queen → [0.27, 0.75, 0.10]

👉 Similar words → similar vectors

Magic Property:

king - man + woman $\approx$ queen

Types:

◆ **Word2Vec**

- Learns from context
- Predicts words

◆ **GloVe**

- Uses global statistics
- Based on co-occurrence

◆ **FastText**

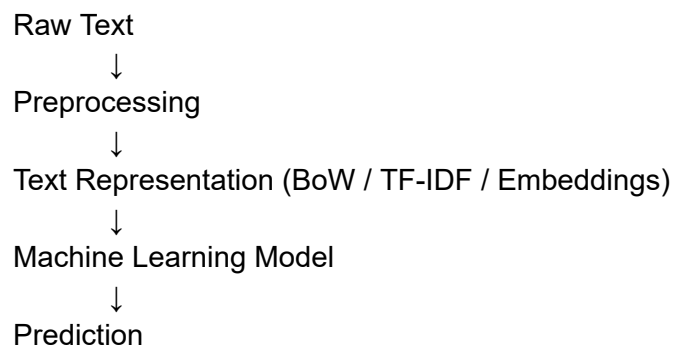
- Considers subwords (great for unknown words)
-

Analogy:

Words placed in a **meaning space** 
(similar words are close together)

4. How It All Fits Together

Flow:



5. When to Use What?

Technique	Use Case	Pros	Cons
BoW	Simple tasks	Easy	No context
TF-IDF	Search, ranking	Better importance	Still no deep meaning
N-Grams	Phrase detection	Captures order	High dimensional
Embeddings	Advanced NLP	Semantic meaning	More complex

Final Mental Model

 Text Representation = Turning language into **math + meaning**

PRACTICAL IMPLEMENTATION — Text Representation & Feature Engineering

Now we'll implement **ALL techniques step-by-step** in Python 🔥
This is exactly what you'll explain in interviews.



1. Bag of Words (BoW)

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "I love NLP",
    "I love Machine Learning",
    "NLP is amazing"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("BoW Matrix:\n", X.toarray())
```

Explanation

- `CountVectorizer()` → converts text → word counts
- `fit_transform()` → learns vocabulary + creates matrix

👉 Output (example):

```
Vocabulary: ['amazing', 'is', 'learning', 'love', 'machine', 'nlp']
BoW Matrix:
[[0 0 0 1 0 1]
 [0 0 1 1 1 0]
 [1 1 0 0 0 1]]
```

What's happening?

Each row = sentence
Each column = word
Value = frequency



2. TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print("TF-IDF Matrix:\n", X.toarray())
```



Explanation

- Same as BoW but:
 - Reduces weight of common words
 - Highlights important words



Key Insight

- "love" appears everywhere → lower weight
- "amazing" appears once → higher weight



3. N-Grams

```
vectorizer = CountVectorizer(ngram_range=(1,2)) # Unigram + Bigram
X = vectorizer.fit_transform(corpus)

print("N-grams:", vectorizer.get_feature_names_out())
```



Explanation

- (1,2) → include:
 - single words
 - pairs of words



Example Output

['i', 'love', 'nlp', 'i love', 'love nlp', ...]

💡 Why useful?

Captures meaning like:

- "not good" vs "good"
-



4. Word Embeddings (Word2Vec)

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize

sentences = [word_tokenize(text.lower()) for text in corpus]

model = Word2Vec(sentences, vector_size=50, window=3, min_count=1)

print(model.wv['nlp']) # vector for word "nlp"
```

🔍 Explanation

- `Word2Vec()` → learns word relationships
 - `vector_size=50` → size of vector
 - `window=3` → context size
-

💬 Magic Feature

`model.wv.most_similar('nlp')`

👉 Returns similar words

5. Real-World Use Cases (VERY IMPORTANT)

Search Engines

- TF-IDF for ranking documents
-

Chatbots / RAG Systems (YOU SHOULD FOCUS HERE)

- Embeddings used for:
 - semantic search
 - similarity matching
-

Healthcare (Your Domain)

- Clinical text similarity
 - Symptom clustering
-

Data Engineering Pipeline

S3 → Preprocessing → Embeddings → Vector DB → LLM → Response

7. Common Mistakes

1. Using BoW for complex problems

 No context understanding

✖ 2. Ignoring preprocessing

👉 Garbage input = garbage vectors

✖ 3. Wrong n-gram size

👉 Too large = huge memory

✖ 4. Not normalizing vectors

👉 Affects similarity search

✖ 5. Using Word2Vec instead of modern embeddings

👉 Prefer:

- Sentence Transformers
 - OpenAI embeddings
-



Final Practical Understanding

Text → Clean → BoW / TF-IDF / Embeddings → Model → Prediction
